

Lezione 19 - Python e array

NUMPY E ARRAY

◆ Idea generale

👉 In Python possiamo lavorare con array usando la libreria **numpy**

👉 Numpy è una libreria per:

- Lavorare con array
- Fare operazioni numeriche efficienti

✳ Fondamentale per il calcolo scientifico e parallelo

◆ Perché usare numpy

👉 Numpy:

- E' molto veloce
- Usa strutture dati compatte
- E' base per altre librerie

👉 Esempi:

- pandas
- PyTorch
- TensorFlow

◆ Vantaggi principali

👉 Permette di:

- Gestire dati in modo efficiente
- Passare dati ad altre librerie

- Lavorare meglio con GPU

✳️ Molto usato in ambito scientifico e machine learning

Ora vediamo il confronto tra array numpy e liste Python

NUMPY ARRAY VS LISTE PYTHON

Differenza strutturale

👉 Le liste Python:

- Sono simili a tabelle hash
- Possono contenere tipi diversi
- Usano più memoria del minimo necessario

👉 Gli array numpy:

- Sono simili a vettori in C
- Hanno dati contigui in memoria
- Sono più compatti

✳️ Organizzazione della memoria più efficiente

Prestazioni

👉 Liste Python:

- Append veloce
- Ma meno efficienti nelle operazioni numeriche

👉 Array numpy:

- Più veloci se facciamo operazioni su tutti gli elementi
- Più lenti se modifichiamo continuamente la loro dimensione

◆ Tipi di dati

👉 Numpy funziona meglio con:

- tipi omogenei (tutti int, tutti float, ecc.)

👉 mentre le liste:

- Possono contenere tipi diversi

✳ Numpy è ottimizzato per calcolo numerico

◆ Riassumendo

👉 Numpy è ideale quando:

- Lavoriamo con molti dati
- Facciamo operazioni matematiche

👉 Le liste sono migliori quando:

- Hanno struttura dinamica
- Hanno dati eterogenei

Ora vediamo come creare array (base) in numpy

📌 CREAZIONE DI ARRAY (BASE)

◆ Creazione con np.array

👉 Possiamo creare un array numpy usando:



Codice:

```
a = np.array([1, 2, 3])
```

- 👉 Crea un array con valori interi
- 👉 Numpy assegna automaticamente il tipo
- ✳️ dtype = int64



Codice:

```
a = np.array([1., 2., 3.])
```

- 👉 Presenza del punto → numeri float
- 👉 Numpy usa numeri in virgola mobile
- ✳️ dtype = float64



Codice:

```
a = np.array([1., {}, "hello"])
```

- 👉 Elementi di tipo diverso
- 👉 Numpy NON può usare un tipo unico numerico
- ✳️ dtype = object

◆ Osservazione

- 👉 Numpy cerca sempre di:
 - usare un tipo unico per tutti gli elementi
- 👉 se non è possibile:

- usa object

✳️ meno efficiente

Ora vediamo i tipi di dati in numpy

TIPI NUMPY (DTYPE)

◆ Idea

👉 In numpy ogni array ha un tipo associato

👉 Questo tipo si chiama:

✳️ **dtype**

◆ Perché è importante

👉 Il tipo:

- Determina come sono salvati i dati in memoria
- Influenza le prestazioni

✳️ Tipi omogenei = Più velocità

◆ Tipi principali

👉 Tipi interi:

- Con segno: **int8, int16, int32, int64**
- Senza segno: **uint8, uint16, uint32, uint64**

👉 Tipi floating point:

- float16
- float32
- float64

◆ Osservazione

👉 numpy ha una *gerarchia di tipi*

👉 In generale:

- Usare lo stesso tipo per tutti gli elementi

✳️ migliore efficienza

Ora vediamo le proprietà degli array

📌 PROPRIETÀ DEGLI ARRAY

◆ Accesso alle informazioni

👉 Dato un array `a`, possiamo ottenere informazioni usando:

💻 Codice:

```
a.dtype
```

👉 Restituisce il **tipo degli elementi**

💻 Codice:

```
a.shape
```

👉 Restituisce una tupla con le **dimensioni dell'array**

👉 Esempio:

- vettore → una dimensione
- matrice → due dimensioni

✳ es: (4, 3)



Codice:

```
a.size
```

👉 Restituisce il **numero totale di elementi**

✳ es: matrice 4x3 → 12 elementi



Codice:

```
a.astype(nome_tipo)
```

👉 Permette di **convertire il tipo** dell'array

👉 Esempio:

- da int a float
- da float a int

◆ Osservazione

👉 Queste proprietà servono per:

- Capire la struttura dell'array
- Controllare i dati

✳ Fondamentali per lavorare con numpy

Ora vediamo come creare degli array più avanzati

CREAZIONE DI ARRAY (AVANZATA)

◆ Array con valori di default



Codice:

```
a = np.zeros(3)
```

👉 Crea un array di zeri

✳ dtype = float64



Codice:

```
a = np.zeros(3, dtype=np.int16)
```

👉 Crea un array di zeri specificando il tipo

✳ dtype = int16



Codice:

```
a = np.ones(3)
```

👉 Crea un array di uno

✳ dtype = float64



Codice:

```
a = np.full(3, 5.2)
```

👉 Crea un array riempito con lo stesso valore

✳️ `dtype = float64`



Codice:

```
a = np.empty(3)
```

👉 Crea un array *non inizializzato*

✳️ I valori presenti in memoria possono essere qualsiasi cosa

◆ Copiare la forma

👉 Viene copiata la forma del vettore passato come argomento



Codice:

```
a = np.zeros(3)
b = np.ones_like(a)
```

👉 `b` ha la stessa *shape* di `a`

👉 Cambia solo il contenuto

👉 Esistono anche:

- `zeros_like`
- `empty_like`
- `full_like`

◆ Creazione con range

👉 Posso riempire un array usando i valori in un range:



Codice:

```
np.arange(6)
```

👉 Crea valori da 0 a 5



Codice:

```
np.arange(1, 6)
```

👉 Crea valori da 1 a 5



Codice:

```
np.arange(1, 6, 2)
```

👉 Crea valori con passo 2

👉 Risultato: 1, 3, 5



Codice:

```
np.linspace(0, 6, 4)
```

👉 Crea 4 valori equispaziati tra 0 e 6

✳ Con `arange` bisogna fare attenzione quando si usano i *float*

◆ Numeri casuali

👉 La "modalità nuova" in numpy prevede due passaggi:

- Inizializzare un generatore
- Usare il generatore per creare i valori casuali



Codice:

```
rng = np.random.default_rng()
```

👉 Inizializza il generatore di numeri pseudocasuali



Codice:

```
rng.integers(a, b, n)
```

👉 Genera interi uniformi in $[a, b)$



Codice:

```
rng.integers(a, b, n, endpoint=True)
```

👉 Genera interi uniformi in $[a, b]$



Codice:

```
rng.uniform(a, b, n)
```

👉 Genera numeri float uniformi in $[a, b)$



Codice:

```
rng.random(n)
```

👉 Genera numeri float uniformi in $[0, 1)$



Codice:

```
rng.standard_normal(n)
```

👉 Genera valori con distribuzione normale standard

✳ $\mu = 0, \sigma = 1$



Codice:

```
rng.normal(a, b, n)
```

👉 Genera valori con distribuzione normale

✳ $\mu = a, \sigma = b$

◆ Perché usarla?

👉 Questa nuova modalità è:

- Più flessibile
- Migliore per il *multithreading*
- statisticamente più solida

Ora vediamo **indicizzazione** e **viste**

INDICIZZAZIONE E VISTE

◆ Indicizzazione base

👉 Possiamo accedere agli elementi di un array come con le liste Python

Codice:

```
a[i]
```

👉 Accede a un singolo elemento

Codice:

```
a[i:j]
```

👉 Prende un range di indici tra *i* (incluso) e *j* (escluso)

Codice:

```
a[start:end:step]
```

👉 Range di indici, con specifica anche del passo



Codice:

```
a[::2]
```

👉 Prende un elemento ogni 2

✳️ Esempio: solo indici pari

◆ Differenza con le liste

👉 Nelle liste Python:

- Il slicing crea una copia

👉 In numpy:

✳️ NON viene creata una copia

◆ Viste (view)

👉 In numpy il risultato del slicing è una:

✳️ *vista*

👉 Una vista è:

- Un riferimento all'array originale
- NON una copia dei dati

👉 Possiamo immaginarla come:

- un puntatore con nuovi indici

◆ Effetto importante

👉 Se modifichiamo la vista:

✳️ modifichiamo anche l'array originale



Codice:

```
b = a[1:3]
```

👉 b è una vista di a

👉 modificando b :

- cambia anche a

◆ Copia esplicita

👉 Se vogliamo una copia vera:



Codice:

```
b = a.copy()
```

👉 Crea un nuovo array indipendente

✳️ modifiche non influenzano l'originale

Ora vediamo indicizzazioni più avanzate (fancy indexing e condizioni)

FANCY INDEXING E CONDIZIONI BOOLEANE

◆ Fancy indexing

👉 Possiamo indicizzare un array usando un altro array di indici

Codice:

```
A = np.array([6, 7, 3, 4, 8])  
B = A[[0, 2, 3]]
```

👉 B contiene:

- A[0] , A[2] , A[3]

👉 Risultato: [6, 3, 4]

◆ Osservazione

👉 In questo caso:

✳️ viene creata una copia

👉 NON è una vista

◆ Maschere booleane

👉 Possiamo usare array di **Booleani** per selezionare elementi

Codice:

```
A[[True, True, False, False, True]]
```


👉 Seleziona solo gli elementi con `True`

👉 Risultato: `[6, 7, 8]`

◆ Condizioni sugli array

👉 Possiamo applicare condizioni direttamente:



Codice:

```
B = A > 4
```

👉 Restituisce un array di Booleani

👉 Esempio: `[True, True, False, False, True]`



Codice:

```
A[A > 4]
```

👉 Seleziona solo i valori che soddisfano la condizione

👉 Risultato: `[6, 7, 8]`

◆ Combinare condizioni

👉 Possiamo combinare condizioni con operatori logici:

- `&` → and
- `|` → or
- `^` → xor
- `~` → not



Codice:

```
(A > 4) & (A < 8)
```

👉 Seleziona valori tra 4 e 8

◆ Verifiche globali

👉 Possiamo verificare condizioni su tutto l'array:



Codice:

```
np.any(condizione)
```

👉 Vero se *almeno un elemento* soddisfa la condizione



Codice:

```
np.all(condizione)
```

👉 Vero se *tutti gli elementi* soddisfano la condizione

✳ Strumenti fondamentali per filtrare dati

Ora vediamo operazioni utili con condizioni (clip e where)

CLIP E WHERE

◆ Limitare valori con clip

👉 Possiamo limitare i valori di un array in un intervallo

Codice:

```
np.clip(a, min, max)
```

👉 Comportamento:

- se un valore $< \text{min}$ $\rightarrow$ diventa min
- se un valore $> \text{max}$ $\rightarrow$ diventa max

✳ Tutti i valori restano nel range $[\text{min}, \text{max}]$

◆ Trovare indici con where

👉 Possiamo trovare dove una condizione è verificata

Codice:

```
np.where(A > 5)
```

👉 Restituisce gli **indici** degli elementi che soddisfano la condizione

👉 risultato:

- una tupla
- con array di indici

◆ Osservazione

👉 `where` NON restituisce i valori

👉 ma le posizioni degli elementi

✳️ utile per lavorare sugli indici

Ora vediamo le funzioni e i metodi principali di numpy

📌 FUNZIONI E METODI NUMPY

◆ Funzioni su array

👉 Numpy mette a disposizione molte **funzioni matematiche** applicabili agli array



Codice:

```
np.sqrt(A)
```

👉 Radice quadrata



Codice:

```
np.exp(A)
```

👉 Esponenziale



Codice:

```
np.log(A)
```

👉 Logaritmo naturale



Codice:

```
np.sin(A)
```

👉 seno



Codice:

```
np.arcsin(A)
```

👉 Arco seno



Codice:

```
np.arctan(A)
```

👉 Arcotangente



Codice:

```
np.floor(A)
```

👉 Arrotonda per difetto



Codice:

```
np.ceil(A)
```

👉 Arrotonda per eccesso



Codice:

```
np.round(A)
```

👉 Arrotondamento



Codice:

```
np.dot(A, B)
```

👉 Prodotto interno



Codice:

```
A @ B
```

👉 Equivalente a dot



Codice:

```
np.sort(A)
```

👉 Restituisce una copia ordinata

◆ Metodi degli array

👉 Gli array hanno anche **metodi propri**



Codice:

```
A.sum()
```

👉 Somma degli elementi



Codice:

```
A.mean()
```

👉 Media



Codice:

```
A.var()
```

👉 Varianza



Codice:

```
A.std()
```

👉 Deviazione standard



Codice:

```
A.min()
```

👉 Valore minimo



Codice:

```
A.max()
```

👉 Valore massimo



Codice:

```
A.argmax()
```

👉 Indice del massimo



Codice:

```
A.argmin()
```

👉 Indice del minimo

◆ Osservazione

👉 Tutte queste operazioni:

- lavorano direttamente sugli array
- NON richiedono cicli espliciti

✳ codice più semplice e veloce

Ora passiamo al parallelismo in Python (GIL e multiprocessing)

GIL E MULTIPROCESSING

◆ Cos'è il GIL

👉 Il **GIL** (*Global Interpreter Lock*) è un meccanismo di Python che:

- Protegge l'accesso agli oggetti
- Gestisce l'esecuzione del codice

✳ Solo un thread alla volta può eseguire codice Python

◆ Effetto del GIL

👉 Anche usando più thread:

- Il codice Python NON viene eseguito in parallelo

✳ Niente vero parallelismo con thread

◆ Quando NON è un problema

👉 Alcune operazioni possono comunque essere parallele:

- Operazioni di I/O
- Funzioni scritte in C (es. **numpy**)

✳ Il GIL non blocca tutto

◆ Problema principale

👉 Se eseguiamo:

- codice Python puro

✳ non possiamo sfruttare più CPU

◆ Soluzione: multiprocessing

👉 Per superare il problema usiamo:

✳ **multiprocessing**

👉 Idea:

- invece di thread → usiamo **processi**

👉 perché:

- Ogni processo ha il suo GIL
- Ogni processo ha il suo interprete

✳ Vero parallelismo

◆ Librerie

👉 Possiamo usare:

- Modulo **multiprocessing** (integrato in Python)
- Come libreria esterna esiste **joblib**

◆ Cosa permette multiprocessing

👉 Permette di:

- creare processi
- Eseguire codice in parallelo
- Gestire comunicazione tra processi

Ora vediamo come creare e gestire i processi (Process)

PROCESS

◆ **Idea**

👉 Con **multiprocessing** possiamo creare processi separati

👉 Ogni processo:

- Esegue una funzione
- Lavora in parallelo

✳️ Ogni processo ha il suo interprete Python

◆ **Creazione di un processo**

Codice:

```
p = mp.Process(target=f, args=(i,))
```

👉 target :

- Funzione da eseguire

👉 args :

- Argomenti della funzione (tupla)

👉 Il processo:

- Viene creato
- Ma NON è ancora attivo

◆ **Avvio del processo**



Codice:

```
p.start()
```

👉 Avvia l'esecuzione del processo

◆ **Attendere la fine**



Codice:

```
p.join()
```

👉 Aspetta che il processo finisca

◆ Esempio completo



Codice:

```
import multiprocessing as mp
import time
import os

def f(n):
    time.sleep(2)
    print(f"{os.getpid()} ha ricevuto il valore numero {n}")

def main():
    procs = []
    for i in range(10):
        p = mp.Process(target=f, args=(i,))
        p.start()
        procs.append(p)
    for p in procs:
        p.join()

if __name__ == '__main__':
    main()
```

◆ Spiegazione

👉 `mp.Process(...)` :

- Crea un processo per ogni `i`

👉 `p.start()` :

- Avvia il processo
- Esegue `f(i)`

👉 `procs.append(p)` :

- Salva i processi in una lista

👉 `p.join()` :

- Aspetta che tutti i processi finiscano

👉 `os.getpid()` :

- Stampa l'ID del processo

✳️ Ogni processo ha un PID diverso

◆ Osservazione

👉 I processi:

- Lavorano in parallelo
- NON condividono memoria automaticamente

✳️ Servono strumenti per comunicare

Ora vediamo come comunicare tra processi (Queue)

📌 COMUNICAZIONE E SINCRONIZZAZIONE TRA PROCESSI

◆ Queue

👉 Una **Queue** è una coda per comunicare tra processi

👉 Permette di:

- Inviare dati
- Ricevere dati

✳️ Comunicazione sicura tra processi



Codice:

```
q = mp.Queue()
```

👉 Crea una coda condivisa



Codice:

```
q.put(n)
```

👉 Inserisce un valore nella coda



Codice:

```
q.get()
```

👉 Prende un valore dalla coda

◆ Esempio



Codice:

```
import multiprocessing as mp
import time
import os

def f(q, n):
    time.sleep(2)
    print(f"{os.getpid()} mette in coda il numero {n}")
    q.put(n)

def main():
    procs = []
    q = mp.Queue()
    for i in range(10):
        p = mp.Process(target=f, args=(q, i))
        p.start()
        procs.append(p)
    for i in range(10):
        n = q.get()
        print(f"ricevuto {n}")
    for p in procs:
        p.join()

if __name__ == '__main__':
    main()
```

◆ Spiegazione

👉 `q.put(n)` :

- Ogni processo inserisce un valore

👉 `q.get()` :

- Il processo principale legge i valori

✳ La Queue gestisce automaticamente la sincronizzazione

◆ Memoria condivisa

👉 Possiamo condividere dati tra processi usando:

- **Value**
- **Array**



Codice:

```
a = mp.Array('i', [0] * 10)
```

👉 Crea un array condiviso

👉 'i' indica il tipo (*intero*)



Codice:

```
a[n] = n
```

👉 Ogni processo scrive nell'array

◆ Esempio



Codice:

```
import multiprocessing as mp
import time
import os

def f(a, n):
    time.sleep(2)
    print(f"{os.getpid()} mette nell'array il numero {n} in posizione {n}")
    a[n] = n

def main():
    procs = []
    a = mp.Array('i', [0] * 10)
    for i in range(10):
        p = mp.Process(target=f, args=(a, i))
        p.start()
        procs.append(p)
    for p in procs:
        p.join()
    for i in range(10):
        print(f"a[{i}] = {a[i]}")

if __name__ == '__main__':
    main()
```

◆ Lock



Un **Lock** serve per evitare accessi simultanei a una risorsa



Codice:

```
lock = mp.Lock()
```

👉 Crea un lock



Codice:

```
lock.acquire()
```

👉 Acquisisce il lock



Codice:

```
lock.release()
```

👉 Rilascia il lock

◆ Osservazione

👉 Serve quando:

- Più processi accedono alla stessa risorsa

👉 In molti casi:

- **Value** e **Array** hanno già un lock interno

✳ Attenzione al **deadlock**:

- Se il lock non viene rilasciato
- Gli altri processi restano bloccati

Ora vediamo un modo più semplice per parallelizzare: Pool

POOL E JOBLIB

◆ Pool

👉 Un **Pool** rappresenta un insieme di processi

👉 Permette di:

- Distribuire automaticamente i task
- Evitare di creare manualmente i processi

✳️ Più semplice dell'uso diretto di `Process`

Codice:

```
p = mp.Pool(processes=4)
```

👉 Crea un pool di 4 processi

Codice:

```
p.map(f, range(10))
```

👉 Applica la funzione `f` a tutti gli elementi

👉 Distribuisce il lavoro tra i processi

◆ Esempio



Codice:

```
import multiprocessing as mp
import time
import os

def f(n):
    time.sleep(2)
    print(f"{os.getpid()} ha ricevuto il valore numero {n}")

def main():
    p = mp.Pool(processes=4)
    p.map(f, range(10))

if __name__ == '__main__':
    main()
```

◆ Osservazione

👉 map :

- Divide automaticamente il lavoro
- Gestisce i processi

✳ Molto comodo per cicli

◆ Joblib

👉 **joblib** è una libreria esterna per parallelismo

👉 E' più semplice da usare per:

- Cicli

- Operazioni ripetitive



Codice:

```
from joblib import Parallel, delayed
```



Codice:

```
Parallel(n_jobs=4)(delayed(f)(i) for i in range(n))
```

👉 `n_jobs=4` :

- Numero di processi

👉 `delayed(f)(i)` :

- Indica che la funzione NON deve essere eseguita subito

✳️ Alternativa più semplice al multiprocessing

Ora vediamo un'ottimizzazione importante: numba (JIT)



NUMBA (JUST-IN-TIME)

♦ Idea

👉 La **JIT** (*Just-In-Time compilation*) è una tecnica che:

- Compila il codice durante l'esecuzione

👉 si spende tempo per compilare:

- ma il codice dopo è più veloce

✳️ Utile se la funzione viene eseguita molte volte

◆ Libreria

👉 In Python possiamo usare:

✳️ **numba**



Codice:

```
from numba import jit
import timeit
import random

@jit
def f(n):
    res = 0
    for i in range(n):
        res += i**2 + 2*i + 1
    return res

def g(n):
    res = 0
    for i in range(n):
        res += i**2 + 2*i + 1
    return res

@jit
def f_slow(lst):
    res = ""
    for x in lst:
        res += str(x)
    return len(res)

def g_slow(lst):
    res = ""
    for x in lst:
        res += str(x)
    return len(res)

print(f"Esecuzione di g: {timeit.timeit(lambda: g(100))}")
print(f"Esecuzione di f: {timeit.timeit(lambda: f(100))}")

lst = [random.random() for _ in range(10)]

print(f"Esecuzione di g_slow: {timeit.timeit(lambda: g_slow(lst))}")
print(f"Esecuzione di f_slow: {timeit.timeit(lambda: f_slow(lst))}")
```


◆ Spiegazione

👉 @jit :

- Compila la funzione in codice nativo

👉 timeit :

- Misura il tempo di esecuzione

👉 f_slow e g_slow :

- Mostrano un caso in cui **numba NON migliora**

✳ Dipende dal tipo di codice

👉 Confronto:

- g → Python normale
- f → versione compilata